

Acoustic Memory

Visualizing the Sonic Identity of Place

Real-Time Fluid Visualization

İpek Erensoy, Arda Eren Ulu · Sound Communication / UPF

Project Overview

Acoustic Memory is an interactive data-art piece that listens to a space, classifies what it hears, and turns that classification into a continuously evolving visual record. Environmental audio captured through a microphone is passed through a Teachable Machine sound model trained on three classes; bird, people, and car and the result is fed into a real-time WebGL fluid simulation. Each detection injects a small bloom of pigment-colored ink into a simulated body of fluid; the ink is then carried, stretched, and folded by the simulation's own velocity field, so that overlapping sounds physically blend into new colors, the way watercolor bleeds into wet paper. Over a session, the canvas accumulates these blooms into a single, continuously mixing sonic portrait of the room.

Conceptual Framework

The system is still not a neutral recorder; it carries a designer's point of view: natural sound (bird) is framed as restorative, social sound (people) as neutral, and mechanical noise (car) as a contributor to cognitive load. This framing is unchanged from the original concept and continues to draw on cognitive load theory, urban noise-pollution research, and restorative-environment theory.

What changed is how that framing is expressed visually. In the original version, each class carried both a semantic shape (a bird silhouette, a human figure, a car icon) and a data point. In the fluid version the figurative layer has been removed entirely: a class is now communicated purely through color and through how that color behaves; how far it spreads, how it interacts with the room's accumulated stress, and how it mixes with colors already on the canvas. This is a deliberate trade-off. The piece loses the immediate legibility of a recognizable icon, but gains a more direct, almost synesthetic mapping between sound and material: colors don't just appear, they bleed, swirl, and combine the way real pigments do, which lets the cognitive-load argument play out visually; a noisy, mechanically dominated room visibly churns and muddies the canvas, while a calm, naturally dominated room settles into clear, separated color.

Model Training: Process and Iterations

The initial training attempt struggled with too few samples per class (8–15), unwanted noise in the recording environment (keyboard clicks, fans, distant traffic), and insufficient variety within each class. In real-time testing, classes blurred together; bird calls would occasionally trigger the people output, and the model's confidence across classes failed to separate cleanly.

To address this, the class list was reduced to bird, people, and car. A previously planned construction class was dropped because its acoustic signature overlapped too heavily with car. Sample counts were substantially increased; 87 for bird, 76 for people, 72 for car and recordings were taken at different times of day in an acoustically cleaner environment to reduce contamination from incidental noise.

After retraining with 50 epochs and an overlap factor of 0.5, the model separated all three classes reliably in real time, resolving the earlier people–bird confusion. This process reinforced that data quality and class balance mattered far more than epoch count.

Visual Engine Redesign: From Discrete Agents to Real-Time Fluid Simulation

Motivation

The original renderer drew each detection as a small figurative sprite, a bird, a human silhouette, a car, orbiting within one of several fixed angular zones, with older sprites fading into a 2D “sediment” trail layer. This made the system's classifications immediately legible, but it also kept the piece closer to an animated infographic than to a

material, painterly object. The team wanted the canvas itself to feel like something sound was physically marking, closer to a wet sheet of paper taking on color, using watercolor-based generative-music interfaces as a visual reference point.

Technical Approach

The renderer was rebuilt from scratch in raw WebGL2. p5.js has been removed entirely from the drawing layer and is no longer a dependency of the project; TensorFlow.js, Teachable Machine, and the Web Audio API are unchanged. The new engine implements a real-time fluid solver based on Jos Stam's Stable Fluids method, extended with vorticity confinement for the swirling, smoke-like motion characteristic of real ink in water:

- Every animation frame, the system solves for an incompressible 2D velocity field; advection, vorticity confinement, divergence, a 22-iteration Jacobi pressure solve, and pressure-gradient subtraction; the same family of techniques used in professional motion-graphics fluid tools.
- Detected sound classes no longer spawn sprites; they inject a small splat of color and a velocity impulse directly into the velocity field. Classification confidence and live input volume control how strong and how frequent these impulses are.
- Color is stored and composited as pigment absorption rather than additive light: each class has an absorption color (white minus its RGB), and overlapping absorption values are summed and subtracted from a paper-colored base. This reproduces the subtractive way real pigments mix, e.g. the teal-green Bird ink and the purple People ink combine toward blue, rather than the additive wash that simple alpha-blended sprites would have produced.
- The canvas background was changed from a dark theme to a warm, lightly grained paper tone, generated procedurally in the display shader, to match the watercolor-on-paper reference aesthetic.
- Velocity and pressure are solved at a low simulation resolution (~144 px) for performance, while the color field itself is advected at a higher resolution (~760 px, capped at 1280 px) using that same velocity field, a standard technique that keeps the simulation affordable without making the color look blocky.

Technical Stack

p5.js · TensorFlow.js · WebGL2 (custom GLSL Stable Fluids solver) · TensorFlow.js · Teachable Machine · Web Audio API